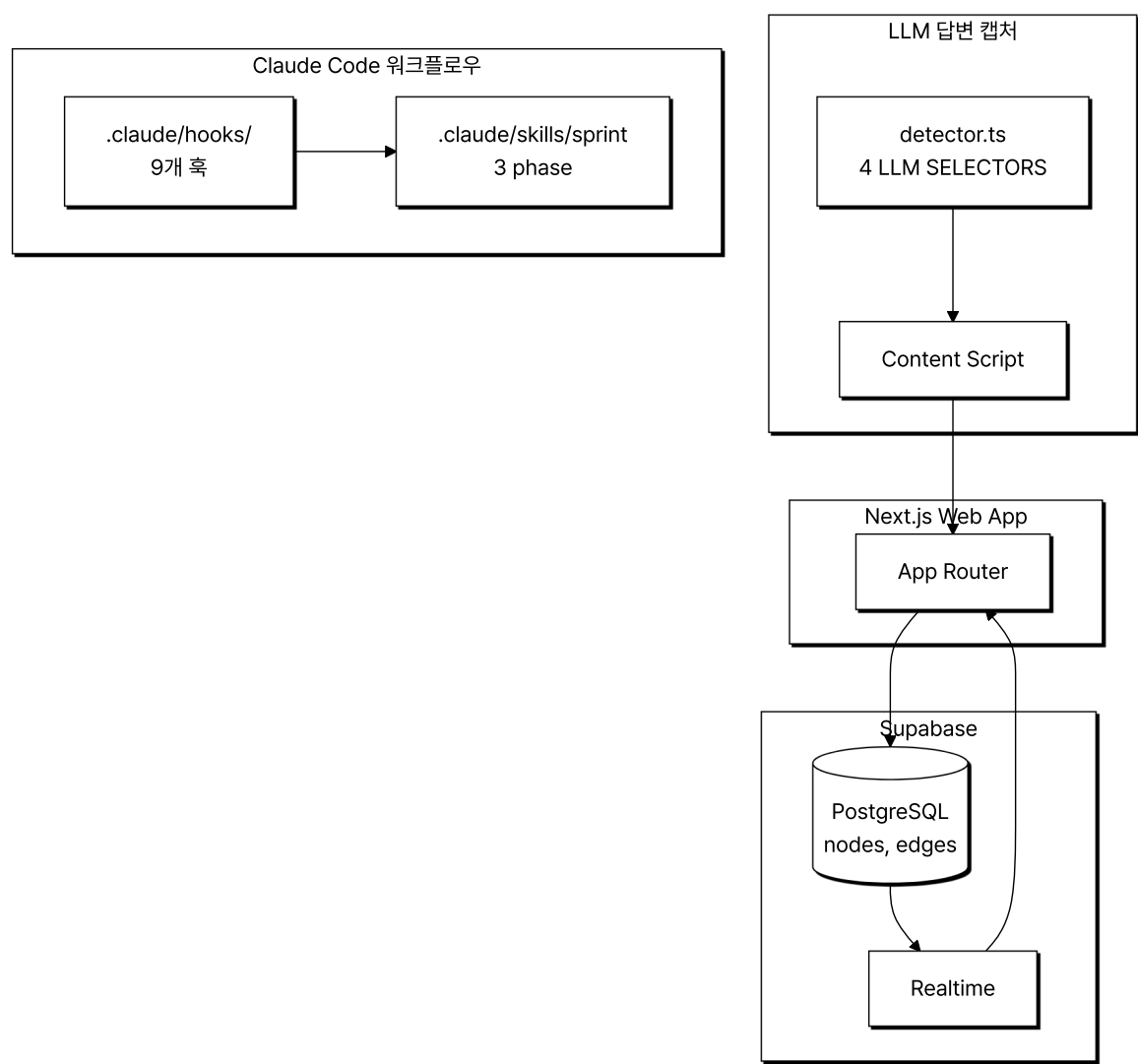


Portfolio

[MINDGRAPH] - AI 지식 캡처 & 그래프 시각화

ChatGPT·Gemini·Claude·Grok 4개 LLM 서비스의 답변을 hostname 자동 감지로 캡처해 동일 스키마(UnifiedNode)로 통합 저장하는 LLM 데이터 도구입니다. Claude Code 위에 9개 훅(hooks)과 plan·qa·ship 3 phase /sprint 워크플로우, 5층 컨텍스트 영속화를 직접 설계해 에이전트가 잘 동작하도록 환경 자체를 코드화한 컨텍스트 엔지니어링·하네스 엔지니어링 사례이며, 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

전체적인 아키텍처



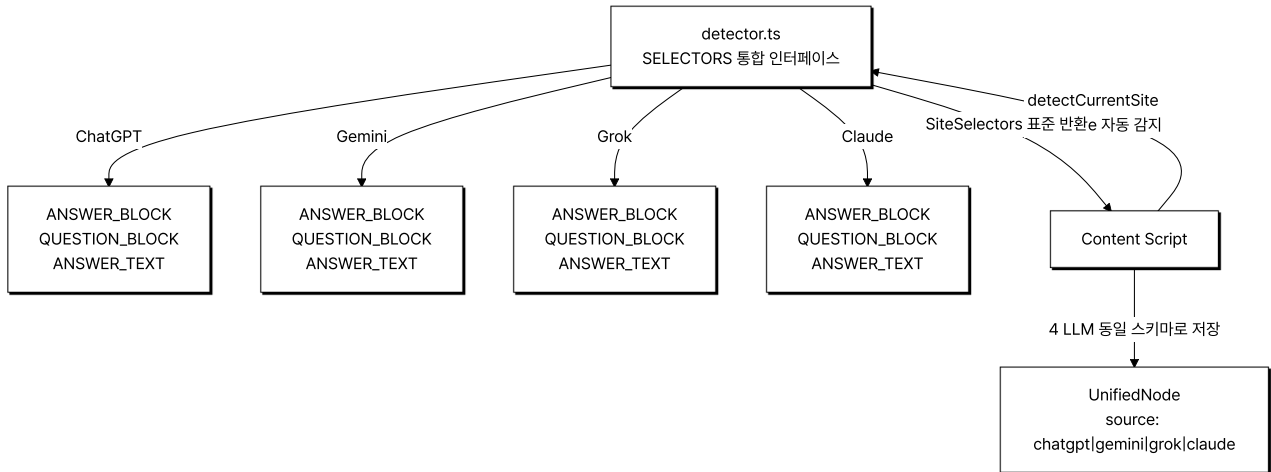
- **Architecture:** LLM 답변 캡처(Extension)와 통합 데이터 저장(UnifiedNode·Supabase), AI 협업 운영(Claude Code 9 훅·3 phase sprint)이 한 시스템 안에서 연결됩니다.

Case 1. 4개 LLM 서비스 통합 표면을 단일 지점으로 추상화

1. 문제 원인

- ChatGPT(OpenAI)·Gemini(Google)·Grok(xAI)·Claude(Anthropic) 4개 LLM 서비스가 각자 다른 응답 DOM 구조를 가지고 있고 사전 공지 없이 자주 바뀌어, 4개 서비스의 응답을 한 시스템에서 통합 캡처하려면 표준 인터페이스로 묶는 통합 레이어가 필요했습니다.
- 초기에는 서비스별 선택자가 여러 파일에 분산되어 한 서비스 DOM 변경이 다수 파일 수정으로 이어졌고, 새 LLM 서비스를 추가하는 비용도 함께 늘었습니다.

2. 해결 과정



- **선택자 통합 인터페이스:** 4개 LLM 서비스의 답변 영역 DOM 선택자를 **detector.ts**의 **SELECTORS** 상수에 **ANSWER_BLOCK · QUESTION_BLOCK · ANSWER_TEXT** 표준 키로 모아, 서비스별 차이를 단일 인터페이스로 추상화했습니다.
- **hostname 자동 감지:** **detectCurrentSite()**가 **window.location.hostname** 기반으로 현재 LLM 서비스를 식별해 올바른 선택자를 반환합니다. 사용자가 서비스를 수동 선택할 필요가 없습니다.
- **UnifiedNode source 필드:** 캡처한 답변은 **source: 'chatgpt' | 'gemini' | 'grok' | 'claude'** 필드를 포함해 동일 스키마로 저장되어, 이후 후속 처리가 LLM 서비스 구분 없이 동작합니다.
- **새 LLM 서비스 추가 경로:** SELECTORS 상수에 새 항목을 추가하면 Content Script·UnifiedNode가 그대로 동작하는 단방향 확장 구조를 유지했습니다.

3. 결과

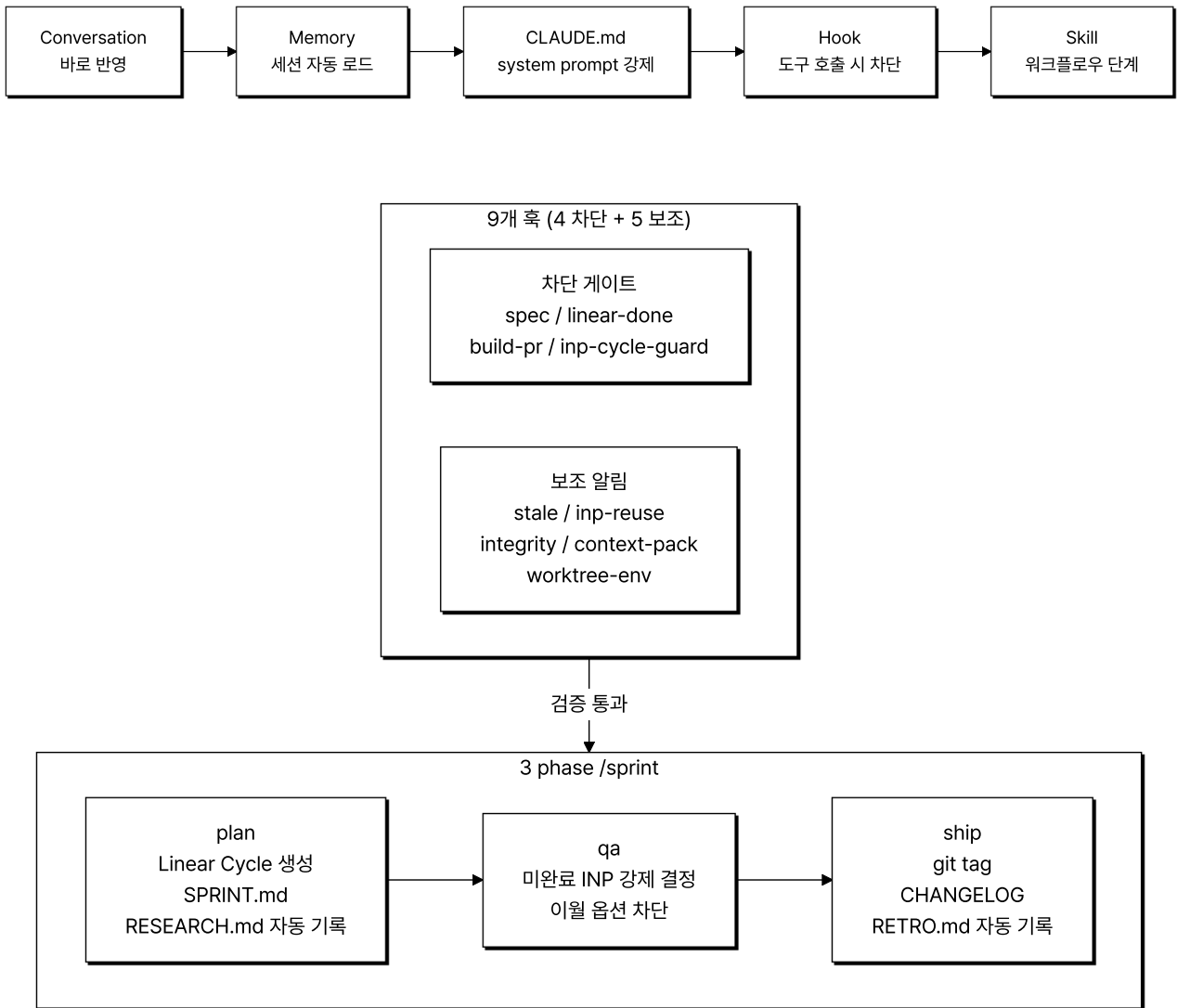
- **성과:** 4개 LLM 서비스의 응답 DOM 변경에 대응할 때 수정 범위가 **detector.ts** 상수 한 곳으로 한정되었고, 새 LLM 서비스를 추가할 때도 SELECTORS 등록 하나로 시스템 전체가 통합되도록 LLM 통합 표면을 단일화했습니다.
- **배운 점:** 다중 LLM 통합 시스템에서는 각 서비스 응답 표현(DOM) 차이를 한 곳에 모아 표준 인터페이스로 추상화하는 통합 레이어 설계가 핵심이고, SELECTORS 상수 한 곳 수정만으로 4개 LLM 서비스 응답 DOM 변경에 대응할 수 있게 되었습니다.

Case 2. 컨텍스트 엔지니어링·하네스 엔지니어링: Claude Code 위 9 혹(하네스) + 3 phase sprint + 5층 컨텍스트 영속화

1. 문제 원인

- Claude Code를 도구로만 쓰면 세션 간 컨텍스트가 초기화되어 같은 규칙·교정을 LLM에게 반복 설명하는 프롬프트 비용이 누적되었고, 이는 LLM 활용 효율을 결정하는 본질 문제였습니다.
- AI가 사용자 확인 없이 commit 생성·이슈 완료 처리·cycle 이월 같은 자율 행동을 해서, LLM 출력 검증을 시스템 수준에서 강제하지 않으면 잘못된 출력이 그대로 main 브랜치에 머지될 위험이 있었습니다.
- 코드·이슈·문서·배포 기록 네 시스템이 같은 정보를 중복 관리해, LLM이 현재 상태를 정확히 파악 못 한 채 응답을 생성하는 컨텍스트 부족 문제가 누적되었습니다.

2. 해결 과정



- **5층 컨텍스트 영속화:** Conversation-Memory-CLAUDE.md-Hook-Skill 5층에 규칙을 분산해, 교정이 바로 반영되고 다음 세션에 자동 로드되며 매 세션 system prompt에 강제되고 도구 호출 시 차단되며 워크플로우 단계로 구조화되도록 했습니다. 이는 LLM 컨텍스트 엔지니어링을 코드 자산으로 정착시킨 사례입니다.
- **9개 혹은 검증 게이트:** 차단 게이트 4개(**spec-gate.sh** 로 스펙 미승인 시 코드 작성 차단, **linear-done-gate.sh** 로 사용자 승인 전 이슈 완료 차단, **build-pr-block.sh** 로 워커 브랜치 PR 차단, **inp-cycle-guard.sh** 로 active cycle 외 이슈 cycle 이동 차단)와 보조 알림 5개(**stale-warn.sh** , **inp-reuse-suggest.sh** , **integrity-sync.sh** , **context-pack-stale.sh** , **worktree-env-symlink.sh**)로 분류해 LLM 출력 검증을 시스템 수준에서 강제합니다.
- **/sprint 메타 스킬 3 phase:** plan(Linear Cycle 생성·SPRINT.md 작성·RESEARCH.md 자동 기록), qa(미완료 INP를 완료 또는 archive로 강제, 이월 옵션 mechanism 차단), ship(git tag·CHANGELOG·RETRO.md 자동 기록)을 단일 명령으로 연결하고, 각 phase 진입 전 이전 산출물 존재 여부를 검증해 LLM에 일관된 컨텍스트가 단계별로 주어지도록 강제합니다.
- **4중 SSOT:** SPRINT.md-Linear Cycle-Git-CHANGELOG 네 시스템에 역할 하나씩만 부여해 LLM이 항상 단일 진실 원천에서 컨텍스트를 받도록 했습니다.
- **부서·에이전트 분리 + context-map 자동 라우팅:** 7개 부서(design·dev·docs·marketing·ops·product·qa)와 9개 에이전트(ceo·frontend/backend/qa-engineer·product-manager·ui-ux-designer·marketing-strategist·ops-engineer·knowledge-logger)별로 **department/{팀}/CLAUDE.md** 와 **docs/ lifecycle** 폴더(active/playbook/artifacts/drafts/archive)를 분리하고, **RULES/context-map.md** 정책 doc과 **SYSTEM/schemas/code-doc-mapping.yaml** 매핑 데이터로 task_type 6종(new_feature·ui_change·api_change·copy_change·bug_fix·refactor) 및 코드 path glob(예: **storage/**** 변경 시 **DATABASE_SCHEMA.md** · **BACKEND.md** · **STATE_MANAGEMENT.md**)을 must-read doc에 자동 매핑했습니다. CEO 에이전트

트가 워커(frontend-engineer-backend-engineer-qa-engineer)에 위임할 때 task_types.must_read와 path_mappings 합집합만 prompt에 첨부되어, 전 부서 docs 끌어오기를 5개 내외 doc 첨부로 줄였습니다. `scripts/docs/validate.ts`의 `validateContextMap()`이 must-read 실재성을 검증해 라우팅이 깨지지 않도록 했습니다.

3. 결과

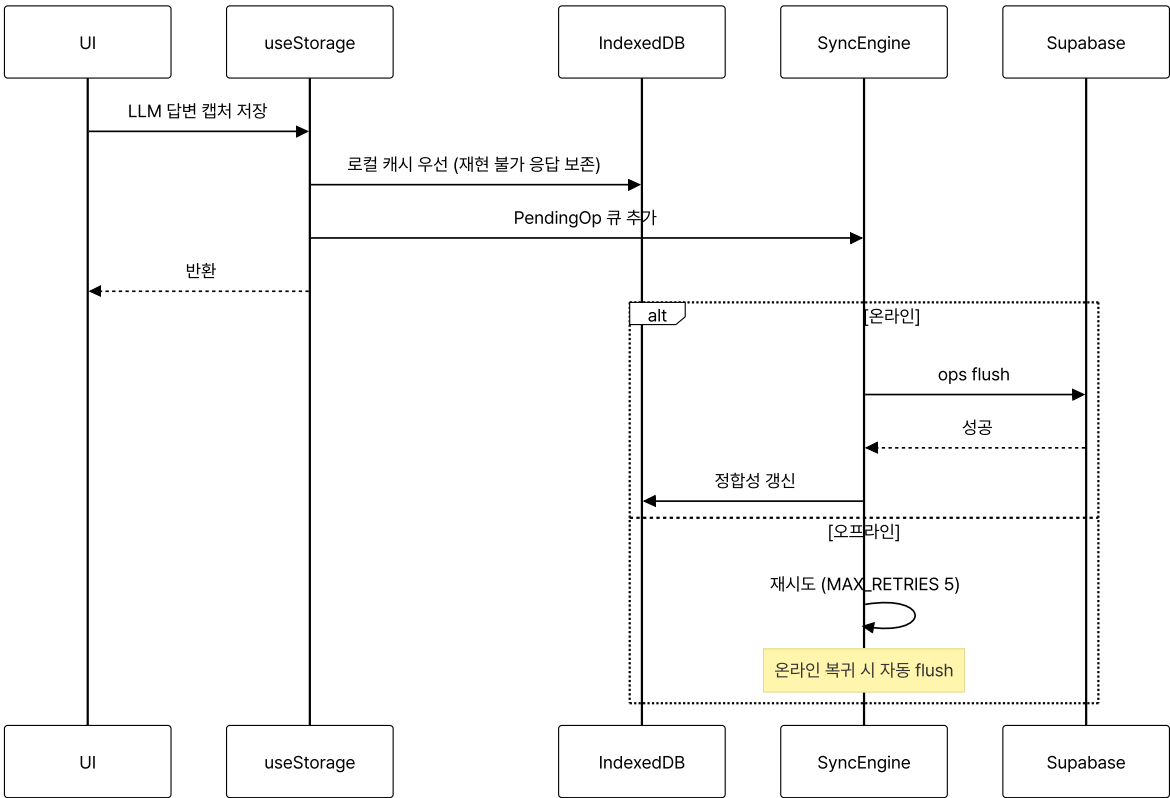
- 성과:** 이슈 등록·SPRINT.md·CHANGELOG·Git 태그·Linear Done 같은 sprint 라이프사이클 반복 작업을 9개 혹과 `/sprint` 3 phase로 자동화해 LLM이 정확한 컨텍스트를 받아 검증된 출력을 만드는 운영 사이클을 코드화했습니다.
- 배운 점:** Conversation-Memory·CLAUDE.md·Hook-Skill 5층과 9개 혹, `/sprint` 3 phase, 4중 SSOT를 코드로 두면서, sprint 라이프사이클 반복 작업과 검증 게이트가 사용자 확인 없이 자율 실행되는 위험을 같은 코드 경로에서 처리하게 되었습니다.

Case 3. LLM 응답 데이터 무손실 동기 파이프라인

1. 문제 원인

- LLM 답변은 모델 응답이라 같은 프롬프트로도 다시 동일하게 재현되지 않을 수 있어, 한 번 캡처한 결과를 잃지 않는 무손실 저장 정책이 LLM 데이터 파이프라인에 필수였습니다.
- LLM 답변이 네트워크 단절로 유실되면 사용자가 다시 LLM에게 같은 프롬프트를 던져 응답을 받아야 했고, 이는 LLM 호출 비용과 사용자 시간을 동시에 낭비하는 데이터 무결성 문제였습니다.
- Supabase RLS 정책 적용 중 오프라인 상태 인증 토큰 만료 변수까지 함께 처리해야 해, 단순 캐시가 아닌 시스템 결정이 필요했습니다.

2. 해결 과정



- 로컬 캐시 우선 저장:** 캡처한 LLM 답변을 IndexedDB에 먼저 쓰는 Write-Behind 방식으로 LLM 호출과 저장을 분리해 재현 불가능한 응답을 네트워크 상태와 무관하게 보존합니다.
- Pending Ops 큐:** `sync-engine.ts`의 `addPendingOps`와 `flushPendingOps`가 동기화 실패를 큐로 흡수하고, 온라인 복귀 시 자동으로 flush됩니다.

- **재시도 한도:** `MAX_RETRIES = 5` 상수로 무한 재시도를 방지하고 초과 시 `status: 'failed'` 로 마킹해 재시도 비용 폭발을 차단했습니다.
- **Realtime 통합:** Supabase Realtime이 다른 디바이스에서 캡처된 LLM 답변을 현재 디바이스 캐시에 자동 반영합니다.
- **캐시 분리:** 클라우드 캐시(`mindgraph-cloud-cache`)와 로컬 데이터(`mindgraph-web`)를 별도 IndexedDB로 두어 로그아웃 시 클라우드로만 비우고 비로그인 사용자 LLM 답변은 로컬에 유지됩니다.

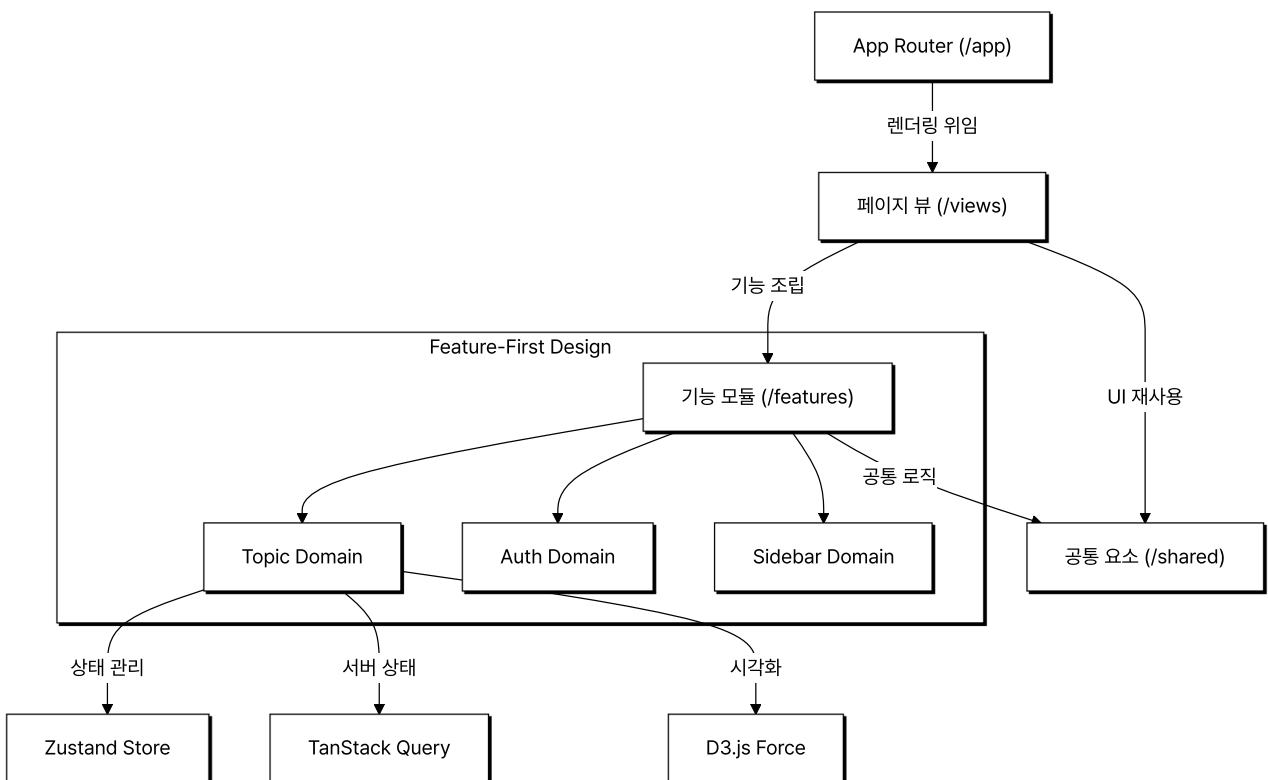
3. 결과

- **성과:** 오프라인 상태에서 LLM 답변 캡처가 동작하고 온라인 복귀 시 큐가 자동 flush되며, 같은 LLM에게 같은 프롬프트로 재요청해야 하는 비용·시간 손실이 사라졌습니다.
- **배운 점:** IndexedDB 우선 저장, Pending Ops 큐(`MAX_RETRIES=5`), Supabase Realtime, 로컬/클라우드 캐시 분리를 묶어, 오프라인 캡처와 온라인 복귀 자동 flush, 다른 디바이스 변경의 자동 반영을 같은 동기 파이프라인에서 처리했습니다.

[CHATGRAPH] - AI 대화 시각화 플랫폼

기존 선형 채팅 UI에서는 한 갈래 흐름만 살아남고 도중에 떠올린 분기 질문과 맥락이 휘발되어, 깊이 있게 사고를 확장하려는 사용자가 이전 맥락을 다시 짜맞춰야 하는 비용이 컸습니다. chatGraph는 비선형 사고 흐름을 보존하고 싶은 학습자와 리서처를 대상으로, AI와의 대화를 꼬리물기 형태의 그래프로 시각화하여 분기와 깊이를 잃지 않고 사고를 확장할 수 있도록 돕는 것을 목표로 했습니다.

전체적인 아키텍처



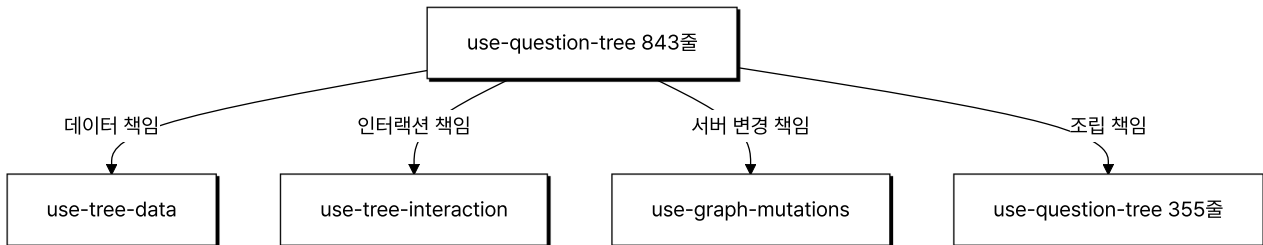
- Next.js App Router 위에 Feature-First 디렉토리 구조를 적용해 도메인 기능과 페이지 조립, 공통 자원을 분리했습니다.
- 본인은 FE 2명 중 1명으로 참여했으며, 아키텍처 재정비, 새 토픽 Optimistic 흐름, `useSuspenseQuery` 기반 데이터 패칭, 뷰 컴포지션 분리, 테스트 코드 작성을 담당했습니다.
- D3 기반 그래프 시각화의 색상 알고리즘과 호버 인터랙션은 팀원이 주도했고, 본인은 React 라이프사이클 통합과 cleanup 로직을 담당했습니다.

Case 1. Feature-First 디렉토리 도입과 거대 혹 분해

1. 문제 원인

- 초기 구조에서는 페이지, 기능, 공통 코드의 경계가 모호하여 컴포넌트끼리 서로를 직접 import하는 순환 참조가 반복 발생했습니다.
- 트리 상태와 인터랙션을 모두 끌어안은 use-question-tree 혹이 843줄까지 비대해져 변경 시 영향 범위 파악이 어려웠습니다.

2. 해결 과정



- app은 라우팅, features는 도메인, views는 조립, shared는 공통이라는 네 가지 책임으로 디렉토리를 재정의하고 import 방향을 단방향으로 정비했습니다.
- 단일 혹이 들고 있던 책임을 트리 데이터(use-tree-data), 인터랙션 상태(use-tree-interaction), 서버 변경(use-graph-mutations)으로 분리하고 최상위에서 합성하는 형태로 정리했습니다.
- 단일 PR이 아닌 본인 단독 커밋 3건에 걸친 누적 리팩토링으로 진행해 위험을 분산했습니다.

3. 결과

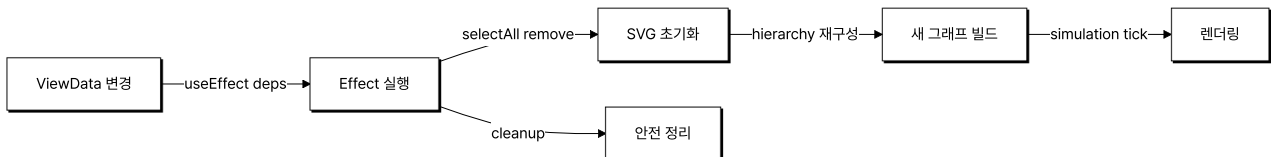
- 핵심 혹을 843줄에서 355줄로 축소하고 인접한 페이지 컴포넌트도 186줄에서 59줄로 정리하여 도메인별 책임을 명확히 했습니다.
- 기능 추가 시 어떤 디렉토리에 어떤 파일이 들어가야 하는지에 대한 팀 내 합의가 잡혀 신규 기능 머지 충돌 빈도가 줄었습니다.
- 디렉토리 재정의 작업을 통해 모듈 간 import 방향을 단방향으로 고정해야 신규 기능이 기존 도메인을 침범하지 않는다는 작업 기준을 팀과 공유했습니다.

Case 2. D3 그래프와 React 라이프사이클의 안전한 통합

1. 문제 원인

- 팀원이 도입한 D3 Force Simulation은 SVG DOM을 직접 변형하기 때문에, React가 동일한 SVG를 다시 그리려고 할 때 잔여 노드나 이벤트 리스너가 남아 시각적 아티팩트가 발생했습니다.
- 토픽이 바뀌거나 트리가 갱신될 때마다 이전 시뮬레이션의 상태가 새 데이터 위에 덧씌워지는 문제가 있었습니다.

2. 해결 과정



- visualizer 컴포넌트의 useEffect에서 `d3.select(svgRef).selectAll("*").remove()` 로 매 갱신마다 SVG 하위 트리를 초기화한 뒤 hierarchy와 simulation을 다시 구성하는 패턴을 정리했습니다.
- onNodeClick과 같이 자주 변하는 콜백은 ref로 감싸 effect 의존성에서 분리하고, currentPath 등 시각적 강조에 영향을 주는 값만 deps에 포함해 불필요한 재구성을 막았습니다.

- 색상 알고리즘과 호버 desaturate 같은 시각적 인터랙션은 팀원이 작성한 코드를 유지하고, 본인은 통합 지점인 라이프사이클과 cleanup 흐름에 집중했습니다.

3. 결과

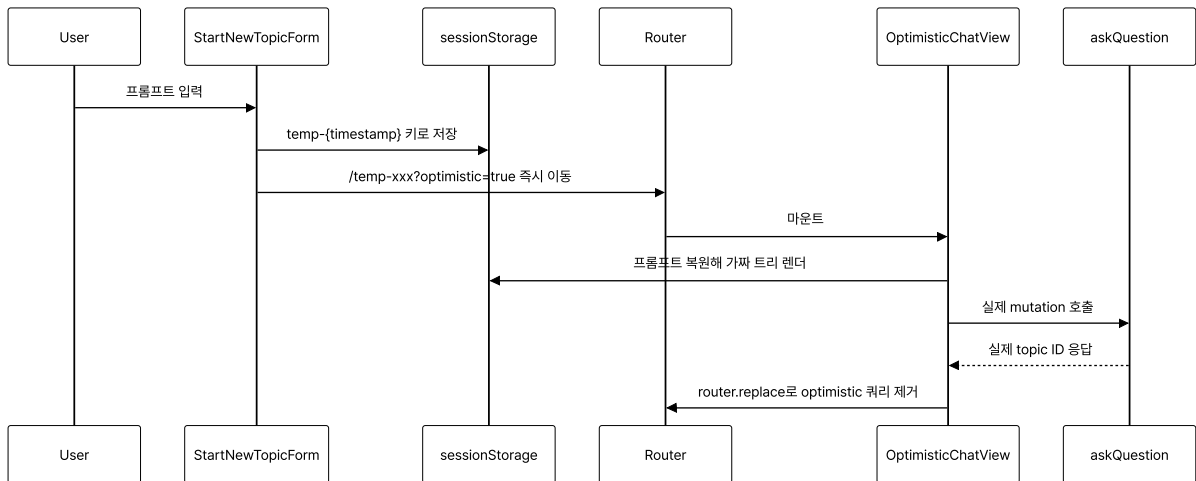
- 토픽이 바뀌거나 트리가 갱신될 때 SVG가 정상적으로 재구성되어 잔여 노드나 중복 이벤트로 인한 시각적 결함이 사라졌습니다.
- React와 D3를 같은 컴포넌트에서 다룰 때 cleanup과 의존성 분리 규칙을 useEffect 한 곳에 모아 두니, 토픽 갱신마다 잔여 DOM이 다음 시뮬레이션을 오염시키는 사고가 더 이상 재현되지 않았습니다.

Case 3. 새 토픽 생성 시 LLM 응답 대기 가리기

1. 문제 원인

- 메인 화면에서 첫 질문을 입력하면 LLM 응답이 도착할 때까지 화면이 정지된 듯한 경험이 사용자 몰입을 끊었습니다.
- 토스트나 스피너만 띄우는 방식은 새 토픽이 생성될 URL을 미리 보여주지 못해 뒤로가기와 새로고침 동작이 어색했습니다.

2. 해결 과정



- StartNewTopicForm에서 temp-{Date.now()} 형태의 임시 ID를 만들어 sessionStorage에 프롬프트를 보관한 뒤 즉시 해당 경로로 push하도록 흐름을 설계했습니다(use-start-new-topic.ts).
- 이동한 경로에서는 useOptimisticTopicData 혹은 sessionStorage 데이터를 복원해 한 단계짜리 트리를 즉시 렌더링하고 동시에 실제 mutation을 실행합니다.
- mutation 성공 시 Zustand 스토어에 응답을 채워 두고 router.replace로 optimistic 쿼리 파라미터를 제거하여 실제 topic ID 경로로 자연스럽게 정착시켰습니다.

3. 결과

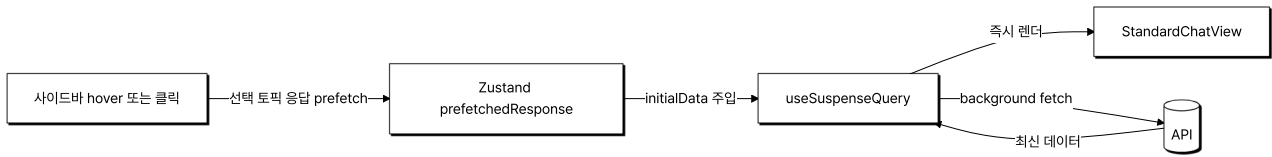
- LLM 응답까지의 대기 시간 동안에도 사용자가 자신이 입력한 질문이 그려진 화면을 즉시 볼 수 있어 체감 대기 시간이 줄었습니다.
- 임시 URL과 실제 URL의 전환을 사용자에게 노출하지 않도록 처리해 새로고침이나 공유 링크 동작도 정상적으로 유지했습니다.
- 라우팅, sessionStorage, mutation 라이프사이클을 한 흐름으로 묶어 두니 LLM 응답까지의 대기 시간이 새 화면 진입 직후 사용자가 자신의 입력을 보는 시간으로 바뀌었습니다.

Case 4. 라우트 전환 시 빈 화면을 없애는 프리패치 패턴

1. 문제 원인

- 사이드바에서 다른 토픽을 클릭하면 useQuery가 새로 호출되며 페이지가 잠시 빈 상태로 보였습니다.
- App Router의 Suspense fallback이 노출되며 깜빡임이 발생했고, 데이터 양이 많은 토픽일수록 체감 지연이 커졌습니다.

2. 해결 과정



- 사이드바에서 토픽 응답을 미리 받아 Zustand 스토어의 `prefetchedResponse` 슬롯에 저장하도록 했습니다.
- 페이지 진입 시 `useSuspenseQuery`의 `initialData`에 동일 `topicId`의 프리패치 응답이 있으면 그대로 사용하고(`use-topic-data.ts`), 없을 때만 Suspense fallback을 거치도록 분기했으며, 다음 단계에서 백엔드 응답을 `transformApiDataToViewData` 어댑터로 트리 형태로 변환해 주입했습니다(`use-tree-data.ts`).
- 사용한 프리패치 데이터는 effect에서 정리해 다음 토픽 전환 시 잔여 데이터가 섞이지 않도록 했습니다.

3. 결과

- 사이드바 기반 라우트 전환에서 빈 화면 노출 없이 트리가 즉시 그려지는 흐름을 확보했습니다.
- TanStack Query 캐시와 Zustand 스토어 두 계층을 모순 없이 결합하는 방법을 익혔고, fallback이 보이는 케이스를 의도적으로 좁히는 설계 감각을 갖게 되었습니다.